Basics Day 9 — Session 9 (Hours 33–36)

Python Programming (Basic) • Functions, Scope & Modules

Session 9 Overview

- Hour 33: Functions — def, parameters, return
- Hour 34: Scope + common function mistakes
- Hour 35: Functions with collections
- Hour 36: Modules — imports and creating utils.py

Hour 33: Functions — def, parameters, return

Learning Outcomes

- Write simple functions using `def`
- Pass arguments and receive parameters
- Return values from functions and store results
- Understand the difference between `return` and `print`

Why Write Functions?

The Problem Without Functions

```
# Without functions — repeated logic everywhere
name1 = input("Name: ").strip().title()
print(f"Hello, {name1}!")

name2 = input("Name: ").strip().title()
print(f"Hello, {name2}!")

name3 = input("Name: ").strip().title()
print(f"Hello, {name3}!")
# Same 2 lines repeated 3 times — hard to update!
```

The Solution — Define Once, Call Many Times

```
def greet(name: str) -> None:
    formatted = name.strip().title()
    print(f"Hello, {formatted}!")
```

Anatomy of a Function

The def Statement

```
def function_name(parameter1, parameter2):
    """Docstring: what this function does."""
    # function body
    return result
```

Breaking It Down

Part	Purpose
`def`	Keyword that starts a function definition
`function_name`	Name you choose — use `snake_case`
`(parameter1, ...)`	Inputs the function receives
`"""..."""`	Docstring explaining the function

Parameters and Arguments

Defining Parameters

```
def add_numbers(a: int, b: int) -> int:
    """Return the sum of a and b."""
    return a + b
```

Calling with Arguments

```
result = add_numbers(3, 5)
print(result)   # 8

print(add_numbers(10, 20))   # 30
print(add_numbers(100, -5))  # 95
```

Positional vs Keyword Arguments

```
def describe_item(name: str, price: float) -> str:
```

return vs print — The Key Difference

print Only Shows on Screen

```
def bad_double(n: int) -> None:
    print(n * 2)    # Only displays; you can't use the value

result = bad_double(5)   # Prints: 10
print(result)            # Prints: None  ← problem!
```

return Sends the Value Back

```
def good_double(n: int) -> int:
    return n * 2    # Sends value back to caller

result = good_double(5)  # result = 10
print(result)            # Prints: 10
total = good_double(5) + good_double(3)   # total = 16
```

> ⚠️ **Key rule:** If you need to use the result later — store it, combine it, pass it on —

Return Values and Storing Results

Functions Are Just Expressions

```
def celsius_to_fahrenheit(c: float) -> float:
    """Convert Celsius to Fahrenheit."""
    return (c * 9 / 5) + 32

# Store the result
boiling = celsius_to_fahrenheit(100)
print(f"100°C = {boiling}°F")     # 100°C = 212.0°F

# Use inline
freezing = celsius_to_fahrenheit(0)
body_temp = celsius_to_fahrenheit(37)

print(f"Freezing: {freezing}°F")   # Freezing: 32.0°F
print(f"Body:     {body_temp}°F")  # Body:     98.6°F
```

A Function Without return Returns None

Demo: Refactor a Calculator Into Functions

Watch For:

- Each function does one job
- Each function returns (not prints) its result
- The `main` block calls the functions and displays results

```
def add(a: float, b: float) -> float:
    """Return a + b."""
    return a + b

def subtract(a: float, b: float) -> float:
    """Return a - b."""
    return a - b

def multiply(a: float, b: float) -> float:
    """Return a * b."""
    return a * b

# Manual testing — call each function and print the result
print("Testing add:")
print(add(3, 4))          # Expected: 7
print(add(-1, 1))         # Expected: 0
```

Lab: Refactor Contact Manager

Instructions (30 minutes)

Task: Refactor a contact manager program by extracting repeated logic into named functions.

Starting Point — messy, repeated code:

```
contacts = {}

# Add a contact
name = input("Name: ").strip().title()
phone = input("Phone: ").strip()
contacts[name] = phone
print(f"Added {name}.")

# List contacts
for name, phone in contacts.items():
    print(f"  {name}: {phone}")
```

Lab: Refactor Contact Manager (continued)

Your Task — extract these three functions:

```python
def add_contact(contacts: dict, name: str, phone: str) -> None:
    """Add a contact to the contacts dictionary."""
    ...  # your code here

def list_contacts(contacts: dict) -> None:
    """Print all contacts."""
    ...  # your code here

def search_contact(contacts: dict, name: str) -> str | None:
    """Return phone number if found, else None."""
    ...  # your code here
```

Requirements:

1. Each function takes `contacts` as its first parameter — no globals
2. `add_contact` stores the entry and prints a confirmation

Common Pitfalls — Hour 33

Pitfall 1: Relying on a Global Variable Unintentionally

```
# Bug: function reads contacts from global scope
contacts = {}

def add_contact(name, phone):
    contacts[name] = phone   # works... but contacts isn't a parameter!
    # If caller passes a different dict, this ignores it
```

```
# Fix: always pass data explicitly
def add_contact(contacts: dict, name: str, phone: str) -> None:
    contacts[name] = phone
    print(f"Added {name}.")
```

Pitfall 2: Returning None by Mistake

```
# Bug: missing return statement
def search_contact(contacts, name):
```

Optional Extensions — Hour 33

Extension 1: Normalize Name on Add

```python
def normalize_name(name: str) -> str:
    """Strip whitespace and title-case a name."""
    return name.strip().title()

def add_contact(contacts: dict, name: str, phone: str) -> None:
    clean_name = normalize_name(name)
    contacts[clean_name] = phone.strip()
    print(f"Added {clean_name}.")
```

Extension 2: Validate Phone Format (Simple)

```python
def is_valid_phone(phone: str) -> bool:
    """Return True if phone is digits only and 7–15 chars."""
    digits_only = phone.replace("-", "").replace(" ", "")
    return digits_only.isdigit() and 7 <= len(digits_only) <= 15
```

Quick Check — Hour 33

Exit Ticket Question: When should a function return a value instead of printing?

Model Answer: "A function should **return** a value when the result needs to be used by the caller — stored in a variable, passed to another function, displayed later, or combined with other values. Use `print` inside a function only when the sole purpose of the function is to display output and the result will never be needed again. As a general rule, prefer `return` — it makes functions more flexible and testable."

Hour 34: Scope + Common Function Mistakes

Learning Outcomes

- Explain local vs global scope in Python
- Avoid unintended reliance on global variables
- Use default parameters for optional behavior
- Handle divide-by-zero with a simple `if` check

What Is Scope?

The Rule

A **variable's scope** is the region of code where it exists and can be used.

Local vs Global

```
greeting = "Hello"        # global — accessible everywhere

def say_hi(name: str) -> None:
    message = f"{greeting}, {name}!"   # message is LOCAL
    print(message)

say_hi("Alice")       # Hello, Alice!
print(message)        # NameError: name 'message' is not defined
```

> 💡 **Local variables live inside the function only.** They are created when the function runs and disappear when it returns.

Local Scope in Detail

Every Function Has Its Own "Space"

```
def compute_area(width: float, height: float) -> float:
    area = width * height    # 'area' is local to compute_area
    return area

def compute_perimeter(width: float, height: float) -> float:
    perimeter = 2 * (width + height)  # different 'area' variable in scope
    return perimeter

print(compute_area(5, 3))        # 15
print(compute_perimeter(5, 3))   # 16
# Neither 'area' nor 'perimeter' exists outside their functions
```

The Scope Lookup Order

Python looks for a variable in this order:

1. **Local** — inside the current function

The Global Variable Trap

A Subtle Bug

```
# Global list — function reads it silently
contacts = ["Alice", "Bob", "Charlie"]

def show_first():
    print(contacts[0])   # reads global 'contacts'

show_first()    # Works... but only with the global!
```

Why This Is a Problem

```
# You want to test with different data
test_contacts = ["Dave", "Eve"]
show_first()   # Still prints "Alice" — ignores test_contacts!
```

The Fix — Pass Data as a Parameter

Demo: Bug Caused by Global, Then Fixed

Watch For:

- First version silently reads global list
- Bug: adding items to wrong list
- Fix: pass the list explicitly

```
# --- Bug Version ---
inventory = []

def add_item(name: str) -> None:
    inventory.append(name)    # silently mutates global!

add_item("Widget")
add_item("Gadget")
print(inventory)   # ['Widget', 'Gadget'] — looks fine...

# What if we wanted a DIFFERENT inventory?
test_inv = []
```

Default Parameters

Making Arguments Optional

```
def greet(name: str, greeting: str = "Hello") -> str:
    """Greet someone with an optional custom greeting."""
    return f"{greeting}, {name}!"

print(greet("Alice"))              # Hello, Alice!
print(greet("Bob", "Good morning"))  # Good morning, Bob!
print(greet("Carol", "Hi"))        # Hi, Carol!
```

Practical Example

```
def format_price(amount: float, currency: str = "USD",
                 decimals: int = 2) -> str:
    """Format a price with optional currency and decimal places."""
    return f"{currency} {amount:.{decimals}f}"

print(format_price(9.99))             # USD 9.99
```

Lab: Calculator Functions

Instructions (30 minutes)

Task: Build a calculator as a set of clean, testable functions, then connect them in a small CLI.

Requirements:

```python
def add(a: float, b: float) -> float:
    """Return a + b."""
    ...

def subtract(a: float, b: float) -> float:
    """Return a - b."""
    ...

def multiply(a: float, b: float) -> float:
    """Return a * b."""
    ...
```

Lab: Calculator Functions (continued)

Build the CLI that uses your functions:

```python
def main() -> None:
    print("Simple Calculator")
    print("Operations: add, subtract, multiply, divide")

    a = float(input("Enter first number: "))
    op = input("Enter operation: ").strip().lower()
    b = float(input("Enter second number: "))

    if op == "add":
        result = add(a, b)
    elif op == "subtract":
        result = subtract(a, b)
    elif op == "multiply":
        result = multiply(a, b)
    elif op == "divide":
        result = divide(a, b)
    else:
        print("Unknown operation.")
        return

    if result is not None:
        print(f"Result: {result}")
```

Common Pitfalls — Hour 34

Pitfall 1: Shadowing Variable Names

```
# Bug: local 'result' shadows intention, but this is fine...
# The real trap is shadowing built-in names
list = [1, 2, 3]     # 'list' now hides the built-in!
print(list([4, 5]))  # TypeError — 'list' is your variable now
```

```
# Fix: never use built-in names as variables
numbers = [1, 2, 3]   # clear, descriptive name
```

Pitfall 2: Not Returning the Computed Value

```
# Bug: computes the answer but throws it away
def multiply(a: float, b: float) -> float:
    product = a * b
    # Missing return!

result = multiply(4, 5)
```

Optional Extensions — Hour 34

Extension: Power and Modulus

```python
def power(base: float, exponent: float) -> float:
    """Return base raised to the power of exponent."""
    return base ** exponent

def modulus(a: int, b: int) -> int | None:
    """Return a mod b, or None if b is zero."""
    if b == 0:
        print("Error: modulus by zero undefined.")
        return None
    return a % b

# Test them:
print(power(2, 10))     # 1024.0
print(power(9, 0.5))    # 3.0 (square root)
print(modulus(17, 5))   # 2
print(modulus(10, 0))   # Error + None
```

Quick Check — Hour 34

Exit Ticket Question: If you create a variable inside a function, can you use it outside?

Model Answer: "No. A variable created inside a function is **local** to that function. It only exists while the function is running and disappears when the function returns. To make a value available outside, you must `return` it from the function and capture it in a variable at the calling scope. Attempting to access a local variable from outside its function raises a `NameError`."

Hour 35: Functions with Collections

Learning Outcomes

- Pass lists and dictionaries into functions
- Decide when to mutate a collection in-place vs return a new one
- Write clear docstrings explaining expectations
- Avoid accidental mutation bugs

Passing Collections to Functions

Lists Are Passed by Reference

```
def double_all(numbers: list) -> None:
    """Double every number in the list in-place."""
    for i in range(len(numbers)):
        numbers[i] = numbers[i] * 2

scores = [10, 20, 30]
double_all(scores)
print(scores)    # [20, 40, 60] — original was modified!
```

The Key Insight

- When you pass a list to a function, **both the caller and the function share the same list object**.
- Changes made inside the function **affect the original list**.

This is called **mutation** — you changed the object in place.

Mutate In-Place vs Return a New Collection

Option A — Mutate In-Place (Modify the Original)

```
def uppercase_names(names: list) -> None:
    """Uppercase all names in the list in-place."""
    for i in range(len(names)):
        names[i] = names[i].upper()

team = ["alice", "bob", "carol"]
uppercase_names(team)
print(team)   # ['ALICE', 'BOB', 'CAROL'] — team is changed
```

Option B — Return a New Collection (Leave Original Alone)

```
def uppercase_names_new(names: list) -> list:
    """Return a new list of uppercased names."""
    result = []
    for name in names:
        result.append(name.upper())
```

When to Choose Which Approach

Prefer Returning a New Collection When:

- You want to keep the original data intact
- The function should be predictable with no side-effects
- You plan to test the function in isolation
- You need both old and new versions at the same time

Prefer Mutating In-Place When:

- The collection is very large (copying would waste memory)
- The intent is clearly "update this structure"
- The function is a "modifier" (e.g., `sort()` in Python's standard library)

Document Your Choice — Use a Docstring

Demo: Normalize a List of Names

Watch For:

- In-place version modifies the original list
- New-list version leaves original untouched
- Print before/after to confirm the difference

```
def normalize_inplace(names: list) -> None:
    """Normalize names in-place: strip whitespace and title-case."""
    for i in range(len(names)):
        names[i] = names[i].strip().title()

def normalize_new(names: list) -> list:
    """Return a new list of normalized names."""
    result = []
    for name in names:
        result.append(name.strip().title())
    return result

raw_names = ["  alice SMITH ", "BOB jones", " carol  "]

# --- In-place ---
copy1 = raw_names.copy()  # copy so we can demo both
```

Lab: Normalize Contacts

Instructions (30 minutes)

Task: Write a function that normalizes a list of contact names and demonstrate it.

Requirements:

```
def normalize_contacts(contacts: list) -> list:
    """Return a new list with each name stripped of whitespace
    and converted to title case.

    The original contacts list is NOT modified.
    """
    ...  # your code here
```

Sample Data to Use:

```
raw_contacts = [
    "  alice smith  ",
```

Lab: Normalize Contacts (continued)

Print Before and After:

```
print("=== Before Normalization ===")
for name in raw_contacts:
    print(f"  {repr(name)}")

cleaned = normalize_contacts(raw_contacts)

print("\n=== After Normalization ===")
for name in cleaned:
    print(f"  {name}")

print("\n=== Original List (should be unchanged) ===")
for name in raw_contacts:
    print(f"  {repr(name)}")
```

Completion Criteria:

- ✅ Names are stripped and title-cased in the returned list

Common Pitfalls — Hour 35

Pitfall 1: Reassigning a Local Variable Without Returning

```
# Bug: local rebind — does NOT modify caller's list
def bad_normalize(names: list) -> None:
    new_list = []
    for n in names:
        new_list.append(n.strip().title())
    names = new_list  # local rebind!
    # 'names' now points to a new list — caller's list is unchanged
    # AND nothing is returned!

data = ["  alice ", "BOB"]
bad_normalize(data)
print(data)   # ['  alice ', 'BOB']  ← unchanged and no return!
```

```
# Fix: return the new list (and capture it at the call site)
def normalize(names: list) -> list:
    result = []
    for n in names:
```

Common Pitfalls — Hour 35 (continued)

Pitfall 2: Modifying a List While Iterating Over It

```
# Bug: removing from a list while looping causes skips
names = ["Alice", "", "Bob", "", "Carol"]

for name in names:
    if name == "":
        names.remove(name)   # modifies list mid-loop!

print(names)   # ['Alice', 'Bob', '', 'Carol'] ← one blank remains!
```

```
# Fix: build a new list with only the items you want
names = ["Alice", "", "Bob", "", "Carol"]
cleaned = []

for name in names:
    if name != "":
        cleaned.append(name)
```

Optional Extensions — Hour 35

Extension: Remove Duplicate Names

```python
def remove_duplicates(contacts: list) -> list:
    """Return a new list with duplicate names removed.

    Case-insensitive: 'Alice' and 'alice' are treated as the same.
    Preserves order of first occurrence.
    """
    seen = set()
    result = []
    for name in contacts:
        key = name.strip().lower()
        if key not in seen:
            seen.add(key)
            result.append(name.strip().title())
    return result

contacts = ["Alice", "alice", "BOB", "Alice", "Carol", "bob"]
unique = remove_duplicates(contacts)
```

Quick Check — Hour 35

Exit Ticket Question: What's one advantage of returning a new list instead of modifying in place?

Model Answer: "Returning a new list preserves the original data unchanged, which means: (1) you can compare before and after, (2) the function has no side-effects — it won't accidentally alter data used elsewhere in the program, and (3) the function is easier to test in isolation. This style is sometimes called a **pure function** — same input always produces the same output without modifying anything outside the function."

Hour 36: Modules — Imports and Creating utils.py

Learning Outcomes

- Import from the Python standard library
- Use `import` vs `from ... import`
- Create your own module and import from it
- Avoid common import mistakes (name conflicts, wrong directory)

What Is a Module?

The Idea

A **module** is just a Python file (`.py`) that contains functions, variables, or classes that you can reuse in other programs.

Two Categories

Type	Examples
Standard library	`math` , `random` , `datetime` , `os`
Your own modules	`utils.py` , `contacts.py` , `helpers.py`

Why Use Modules?

- **Reuse:** write a helper once, import it anywhere
- **Organization:** keep related functions together

import vs from ... import

`import module` — Access Through the Module Name

```
import math

print(math.pi)            # 3.141592653589793
print(math.sqrt(16))      # 4.0
print(math.floor(3.7))    # 3
print(math.ceil(3.2))     # 4
```

`from module import name` — Use the Name Directly

```
from math import pi, sqrt

print(pi)          # 3.141592653589793
print(sqrt(16))    # 4.0
# No 'math.' prefix needed
```

Which to Choose?

Standard Library Tour: math, random, datetime

math — Numeric Utilities

```
import math

print(math.pi)          # 3.141592653589793
print(math.sqrt(25))    # 5.0
print(math.pow(2, 8))   # 256.0
print(math.floor(4.9))  # 4
print(math.ceil(4.1))   # 5
```

random — Random Values

```
import random

print(random.randint(1, 6))       # Simulate a dice roll: 1–6
print(random.choice(["a", "b", "c"]))  # Random item from list
print(random.random())            # Float between 0.0 and 1.0
```

Creating Your Own Module

Step 1 — Write utils.py

```python
# utils.py  (put this in the same folder as your main program)

def safe_int(prompt: str) -> int:
    """Prompt the user until a valid integer is entered."""
    while True:
        raw = input(prompt).strip()
        if raw.lstrip("-").isdigit():
            return int(raw)
        print("Please enter a whole number.")

def format_money(value: float, currency: str = "USD") -> str:
    """Return a formatted money string, e.g. 'USD 12.50'."""
    return f"{currency} {value:.2f}"
```

Step 2 — Import in main.py

Demo: Create utils.py and Import Into main.py

Watch For:

- File structure — both files in the same directory
- Running `main.py` (not `utils.py`) from the project root
- What happens if you run from the wrong directory

```
project/
├── main.py
└── utils.py
```

```python
# utils.py
def safe_int(prompt: str) -> int:
    """Prompt until a valid integer is entered."""
    while True:
        raw = input(prompt).strip()
        if raw.lstrip("-").isdigit():
            return int(raw)
```

Common Import Errors and Fixes

Error 1: Running the Wrong File as Main

```
# Wrong: running utils.py directly — there's no main logic there
$ python utils.py
# No output or unexpected behavior

# Fix: always run main.py
$ python main.py
```

Error 2: Filename Mismatch

```
# Project files:
#   project/
#     main.py
#     helpers.py    # but in main.py you wrote:
#
# In main.py:
#     import utils   # ModuleNotFoundError: No module named 'utils'
```

Lab: Create a utils Module

Instructions (30 minutes)

Task: Build a reusable `utils.py` module and use its helpers inside an existing program.

Step 1 — Create `utils.py` with these two functions:

```
# utils.py

def safe_int(prompt: str) -> int:
    """Keep prompting until the user enters a valid integer.
    Returns the integer value."""
    ...  # your code here

def format_money(value: float, currency: str = "USD") -> str:
    """Return a formatted money string.
    Example: format_money(9.5) → 'USD 9.50'"""
    ...  # your code here
```

Lab: Create a utils Module (continued)

Step 2 — Use them in a tip calculator or to-do manager:

```python
# main.py (Tip Calculator example)
from utils import safe_int, format_money

def calculate_tip(subtotal: float, tip_percent: int) -> float:
    """Return the tip amount."""
    return subtotal * (tip_percent / 100)

def main() -> None:
    print("=== Tip Calculator ===")
    cents = safe_int("Enter bill amount in cents: ")
    subtotal = cents / 100
    tip_pct = safe_int("Enter tip percentage (e.g. 15): ")

    tip = calculate_tip(subtotal, tip_pct)
    total = subtotal + tip

    print(f"Subtotal: {format_money(subtotal)}")
    print(f"Tip ({tip_pct}%): {format_money(tip)}")
    print(f"Total:    {format_money(total)}")
```

Common Pitfalls — Hour 36

Pitfall 1: File Named After a Standard Library Module

```
# Bug: you created 'random.py' in your project folder
# Then at the top of main.py you wrote:
import random
print(random.randint(1, 6))
# AttributeError: module 'random' has no attribute 'randint'
# Python found YOUR random.py, which has no randint!
```

```
# Fix: rename your file
# ✅ Good names: utils.py, helpers.py, my_tools.py, contact_utils.py
# ❌ Bad names: random.py, math.py, os.py, json.py, pathlib.py, string.py
```

Pitfall 2: Running utils.py Instead of main.py

```
# Bug: running the module file directly
$ python utils.py   # nothing happens — no main() call here
```

Optional Extensions — Hour 36

Extension: Add a menu_choice Helper

```python
# Add to utils.py

def menu_choice(prompt: str, options: list) -> str:
    """Show a menu and return the user's valid choice.

    Example:
        choice = menu_choice("Select:", ["add", "list", "quit"])
    """
    while True:
        print(prompt)
        for i, option in enumerate(options, start=1):
            print(f"  {i}. {option}")
        raw = input("Enter choice number: ").strip()
        if raw.isdigit():
            index = int(raw) - 1
            if 0 <= index < len(options):
                return options[index]
        print(f"Please enter a number between 1 and {len(options)}.")

# Usage in main.py
from utils import safe_int, format_money, menu_choice

action = menu_choice("What would you like to do?",
```

Quick Check — Hour 36

Exit Ticket Question: What's the difference between `import math` and `from math import sqrt`?

Model Answer: "Both load the `math` module into memory. The difference is how you access its contents. With `import math`, you access functions through the module name: `math.sqrt(16)`. With `from math import sqrt`, you import the name `sqrt` directly into your namespace and call it as just `sqrt(16)` — no prefix needed. The first style is generally preferred because it makes the origin of the function clear. Use `from ... import` when you have a few frequently-used names and the source is obvious from context."

Session 9 Recap

What We Covered Today

Hour	Topic	Key Takeaway
33	Functions — def, parameters, return	Use `return` to send values back; avoid globals
34	Scope + common mistakes	Local variables are local; pass data in via parameters
35	Functions with collections	Choose in-place vs new list deliberately; document it
36	Modules — imports and utils.py	One module = one responsibility; don't shadow stdlib names

Session 9 Key Concepts

The Four Big Ideas

1. Functions are for reuse AND clarity Write a function when you'd otherwise copy-paste logic — or when naming the logic makes code easier to read.

2. Return values, don't just print `print` is for display. `return` is for sending data back to the caller. Prefer `return` — it makes functions more flexible and testable.

3. Scope: local variables stay local Pass data in through parameters. Avoid silently relying on globals — it makes code hard to test and debug.

4. Modules organize and enable reuse Split related helpers into their own `.py` file. `import` them where needed. Never name your file after a standard library module.

Session 9 Common Patterns Reference

Function That Returns a Value

```
def calculate_discount(price: float, percent: int) -> float:
    """Return the discounted price."""
    return price * (1 - percent / 100)

sale_price = calculate_discount(100.0, 20)
print(f"Sale price: ${sale_price:.2f}")   # Sale price: $80.00
```

Function That Takes and Returns a Collection

```
def filter_short_names(names: list, max_len: int = 5) -> list:
    """Return a new list of names shorter than max_len characters."""
    result = []
    for name in names:
        if len(name) <= max_len:
            result.append(name)
    return result
```

Looking Ahead — Day 10

Next Session Builds On Today

Hour 37: String Methods Deep Dive

- `split()`, `join()`, `strip()`, `replace()`, `find()`
- String cleaning pipelines using functions from today

Hour 38: String Formatting + f-strings

- Format specs: `{value:.2f}`, `{value:>10}`, `{value:,}`
- Building formatted reports

Hour 39: Working With Files — Reading

- `open()`, `with` statement, reading lines
- Process file data using the functions you wrote today